

Informe Técnico: SafeDrop Local

Plataforma de Denuncias Anónimas con Cifrado Híbrido y Arquitectura de Conocimiento Cero

Josué Arbulú

Leonardo Huamán

Julio, 2026

Índice

Informe Técnico del Proyecto SafeDrop Local

Resumen Ejecutivo

1. Contexto y Motivación
2. Arquitectura General del Sistema
3. Fundamento Criptográfico
4. Implementación Técnica por Módulo
5. Flujos de Uso Completos
6. Modelo de Amenazas y Riesgos
7. Plan de Respuesta ante Incidentes (resumen)
8. Consideraciones Éticas
9. Análisis de Seguridad Estática (Bandit)
10. Instrucciones de Ejecución
11. Recomendaciones para una Evolución hacia Producción
12. Conclusiones

Informe Técnico del Proyecto SafeDrop Local

Curso: Ética y Seguridad de Datos (DS3031) — Proyecto Final **Docente:** José Carlos Pazos **Grupo de trabajo:** Josué Arbulú, Leonardo Huamán

Resumen Ejecutivo

SafeDrop Local es una plataforma web de denuncias anónimas que resuelve un problema concreto de confianza organizacional: los canales de denuncia tradicionales exigen que el informante confíe ciegamente en que los administradores del sistema, la base de datos y la red no leerán ni filtrarán su reporte. SafeDrop elimina esa necesidad de confianza mediante una **arquitectura de conocimiento cero**, en la que todo el cifrado ocurre en el navegador del denunciante, el servidor jamás ve texto plano, y la llave capaz de descifrar los reportes se fragmenta y se retira físicamente del servidor apenas se genera.

El sistema combina tres bloques criptográficos que trabajan juntos:

1. **Cifrado híbrido AES-256-GCM + RSA-OAEP-2048** para proteger el contenido de cada denuncia y sus adjuntos.
2. **Shamir's Secret Sharing (2-de-3)** para que ningún actor individual —ni siquiera el servidor— posea la llave privada completa capaz de descifrar denuncias.
3. **Una bitácora de auditoría encadenada con SHA-256**, al estilo de una blockchain simplificada, que hace detectable cualquier alteración retroactiva de los registros del sistema.

Es un proyecto académico (curso DS3031, Ética y Seguridad de Datos), implementado como una aplicación funcional completa: backend en FastAPI, frontend en HTML/CSS/JavaScript puro, base de datos SQLite, y un panel adicional que simula ataques para demostrar en vivo las defensas del sistema.

Este informe explica el proyecto en cuatro capas: **por qué existe** (problema y motivación), **qué hace** (arquitectura y flujos), **cómo lo hace** (implementación técnica por módulo) y **qué tan sólido es** (amenazas, limitaciones y recomendaciones).

1. Contexto y Motivación

1.1 El problema que resuelve

En cualquier organización pueden ocurrir fraude, corrupción, acoso laboral o abuso de autoridad. Existen canales éticos formales para denunciar estas conductas, pero en la práctica muchos empleados no los usan por miedo a represalias. La razón técnica detrás de ese miedo es simple: en un sistema de denuncias convencional, cualquiera con acceso administrativo a la base de datos —un administrador de red, un proveedor de hosting, un técnico de soporte— puede leer el contenido de los reportes en texto plano. La confidencialidad depende, en el fondo, de la buena voluntad de personas con privilegios técnicos, no de una garantía matemática.

SafeDrop Local propone una solución distinta: en vez de pedirle al denunciante que confíe en la organización, el sistema hace que la confidencialidad sea una **propiedad matemática del diseño**, no una promesa administrativa.

1.2 Objetivo general

Implementar una plataforma de denuncias anónimas que cifre los reportes y evidencias en el origen (el navegador), de modo que el servidor y la base de datos solo almacenen datos ilegibles, y que además mantenga un registro de auditoría cuya integridad pueda verificarse criptográficamente en cualquier momento.

1.3 Objetivos específicos

- Cifrar el contenido de cada denuncia con AES-GCM de 256 bits **antes** de que salga del navegador del denunciante.
 - Proteger la llave simétrica de cada denuncia envolviéndola con la llave pública RSA-OAEP de 2048 bits de la organización.
 - Evitar que la llave privada RSA —la única capaz de revertir el cifrado— resida completa en el servidor, dividiéndola mediante Shamir's Secret Sharing.
 - Mantener una bitácora de auditoría a prueba de manipulación, encadenada con SHA-256.
 - Controlar el acceso a funciones sensibles mediante autenticación JWT y roles (RBAC), con contraseñas protegidas por Argon2id.
-

2. Arquitectura General del Sistema

2.1 Vista de alto nivel

SafeDrop Local sigue una arquitectura cliente-servidor clásica, pero con una particularidad importante: **el servidor está deliberadamente “ciego”**. Su única función es autenticar usuarios, almacenar blobs cifrados y mantener la bitácora; nunca ve, procesa ni puede recuperar el contenido real de una denuncia.



El sistema define tres roles funcionales, cada uno con su propia interfaz:

Rol	Interfaz	Requiere login	Puede hacer
Denunciante	<code>index.html</code>	No	Enviar un reporte cifrado con adjuntos, sin registrarse
Auditor	<code>auditor.html</code>	Sí (JWT)	Ver la lista de reportes cifrados y descifrarlos localmente aportando 2 de 3 fragmentos de la llave privada
Administrador	<code>admin.html</code>	Sí (JWT)	Gestionar usuarios, verificar la integridad de la bitácora, simular un ataque de manipulación, respaldar/restaurar la base de datos

Además existen dos piezas de apoyo pedagógico: `simulator.html`, un laboratorio interactivo donde se puede jugar con el cifrado híbrido, Shamir’s Secret Sharing y el encadenamiento de hashes de forma aislada; y `docs.html`, que presenta el informe académico y el resultado del análisis de seguridad estática dentro de la propia aplicación.

2.2 Principio de diseño: conocimiento cero

La idea de “conocimiento cero” (zero-knowledge) en este contexto no se refiere a las pruebas criptográficas formales (zk-SNARKs), sino a un principio de diseño más amplio: **el servidor nunca posee la información necesaria para descifrar lo que almacena**. Esto se logra con dos decisiones concretas:

1. Todo el cifrado ocurre en el navegador, usando la API nativa `window.crypto.subtle` (Web Crypto API). El servidor solo recibe Base64 ya cifrado.

2. La llave privada RSA que podría descifrar esos datos se genera una única vez, se fragmenta inmediatamente con Shamir's Secret Sharing, y el archivo completo se elimina del servidor. Ningún fragmento por sí solo sirve para nada.

2.3 Estructura de directorios del proyecto

```
jhihuhuhu/
├─ run.py                # Orquestador: instala dependencias, genera TLS,
│                        #   crea la BD, genera llaves y arranca el servidor
├─ verify_system.py      # Test de integración end-to-end (fuera del navegador)
├─ inspect_db.py         # Utilidad de inspección directa de la base SQLite
├─ requirements.txt      # Dependencias Python
├─ bandit.yaml           # Configuración del análisis estático de seguridad
├─ backend/
│   ├── main.py          # Aplicación FastAPI y todos los endpoints REST
│   ├── database.py      # Modelos ORM (SQLAlchemy) y lógica de la bitácora
│   ├── auth.py          # JWT, Argon2id y control de acceso por rol
│   ├── crypto_utils.py  # Generación de llaves RSA y Shamir's Secret Sharing
│   ├── ssl_gen.py       # Generación de certificados TLS autofirmados
│   ├── keys/org_public.pem # Llave pública RSA de la organización
│   └─ certs/            # Certificado y llave TLS de localhost
├─ frontend/
│   ├── index.html       # Portal de denuncias
│   ├── auditor.html     # Panel del auditor
│   ├── admin.html       # Panel de administración
│   ├── simulator.html   # Laboratorio interactivo de criptografía
│   ├── docs.html        # Informe y reporte SAST embebidos
│   ├── css/style.css
│   └─ js/
│       ├── crypto_client.js # Toda la criptografía del lado del cliente
│       └─ app_client.js    # Lógica de interfaz y llamadas a la API
```

3. Fundamento Criptográfico

Esta sección explica **por qué** se eligió cada algoritmo y **cómo encajan entre sí**. La implementación concreta se detalla en la sección 4.

3.1 Cifrado híbrido: por qué AES y RSA trabajan juntos

RSA es matemáticamente robusto pero computacionalmente costoso: cifrar directamente un archivo grande con RSA es lento e impráctico, además de que RSA solo puede cifrar bloques de datos más pequeños que el tamaño de la llave. AES, en cambio, es extremadamente rápido y puede cifrar cualquier volumen de datos, pero requiere que ambas partes compartan la misma llave secreta de antemano — y transmitir esa llave de forma segura es, en sí mismo, un problema.

El cifrado híbrido combina lo mejor de ambos:

1. El navegador genera una llave AES-256 **nueva y aleatoria para cada denuncia** (una “llave de sesión” de un solo uso).
2. Esa llave cifra el texto del reporte y cada archivo adjunto con AES-GCM.
3. La llave AES —que es pequeña, solo 32 bytes— se cifra a su vez con la llave pública RSA-OAEP de la organización.
4. Se envía al servidor tanto el contenido cifrado como la llave AES ya envuelta en RSA.

De esta manera, RSA no cifra el contenido pesado, solo la pequeña llave que lo protege — resolviendo el problema de intercambio de llaves sin pagar el costo computacional de cifrar todo con RSA.



AES-GCM fue elegido en particular (frente a otros modos como AES-CBC) porque es un modo de **cifrado autenticado**: además de ocultar el contenido, genera una etiqueta de autenticidad que permite detectar si el texto cifrado fue alterado. Si alguien modifica un solo bit del blob almacenado en la base de datos, el descifrado falla de forma explícita en vez de producir datos corruptos silenciosamente.

RSA-OAEP (Optimal Asymmetric Encryption Padding) se usa en vez de un esquema RSA más simple porque OAEP añade aleatoriedad al cifrado y previene ataques conocidos contra el padding tradicional. Con SHA-256 como función hash interna, cumple con las recomendaciones actuales del NIST.

3.2 Shamir's Secret Sharing: eliminar el punto único de confianza

Incluso con cifrado híbrido, queda un problema: alguien tiene que poseer la llave privada RSA para poder leer las denuncias en algún momento. Si esa llave vive completa en el servidor o en manos de una sola persona, se recrea el mismo problema de confianza que el proyecto intenta resolver.

La solución es **Shamir's Secret Sharing (SSS)**, un esquema criptográfico que permite dividir un secreto en n fragmentos de modo que se necesiten al menos k de ellos para reconstruirlo — pero con $k-1$ fragmentos no se obtiene absolutamente ninguna información sobre el secreto original. SafeDrop implementa un esquema **2-de-3**: la llave privada se divide en 3 fragmentos, y se necesitan al menos 2 para reconstruirla.

Matemáticamente, esto se basa en interpolación polinomial sobre un campo finito. Para un umbral de 2, el esquema define una recta $f(x) = S + a \cdot x$ en el campo de Galois $GF(256)$ (los cálculos se hacen byte a byte, no con números reales), donde S es el secreto y a es un coeficiente aleatorio. Cada fragmento es un punto $(x, f(x))$ de esa recta: $x=1$, $x=2$ y $x=3$. Con solo un punto no se puede determinar la recta (pasan infinitas rectas por un punto), pero con dos puntos cualesquiera se puede calcular la recta exacta mediante **interpolación de Lagrange**, y evaluarla en $x=0$ recupera el secreto S .

El uso práctico de esto en SafeDrop: al arrancar el sistema por primera vez, se genera la llave privada RSA, se divide inmediatamente en 3 archivos `.share`, y **la llave completa se elimina del servidor sin dejar rastro**. Cada fragmento por separado no sirve para nada. Para descifrar una denuncia, el auditor debe cargar al menos dos fragmentos distintos en el navegador, que reconstruye la llave privada en memoria, la usa una sola vez, y la descarta.

3.3 Bitácora de auditoría encadenada (hash chain)

El tercer pilar criptográfico no protege la confidencialidad, sino la **integridad histórica** del sistema: que nadie pueda alterar o borrar un registro pasado (por ejemplo, ocultar que alguien inició sesión, o que se manipuló la base de datos) sin que quede evidencia matemática de ello.

El mecanismo es el mismo principio que sostiene a una blockchain, aplicado a una sola tabla: cada registro de auditoría almacena el hash SHA-256 del registro anterior (`previous_hash`) además de su propio hash (`current_hash`), calculado sobre todos sus campos **incluyendo** ese `previous_hash`. Esto forma una cadena donde cada eslabón depende criptográficamente del anterior.

Si alguien edita el contenido de un log antiguo directamente en la base de datos —sin recalcularse toda la cadena de hashes posteriores—, el hash almacenado deja de coincidir con el hash que se recalcula a partir del contenido actual. La verificación de integridad recorre toda la cadena, recalcula cada hash y compara: cualquier discrepancia, en cualquier punto de la cadena, invalida todo lo que viene después.

3.4 Autenticación y control de acceso

Un cuarto componente, complementario a los tres anteriores: proteger *quién* puede hacer *qué* dentro del sistema.

- **Argon2id** protege las contraseñas de administradores y auditores. Es el algoritmo ganador de la Password Hashing Competition y está diseñado específicamente para ser costoso de atacar por fuerza bruta con GPUs o hardware especializado (ASICs), a diferencia de hashes rápidos como SHA-256 puro, que no deberían usarse nunca para contraseñas.
- **JWT (JSON Web Tokens)**, firmados con HS256, gestionan las sesiones autenticadas una vez que el usuario inicia sesión.
- **RBAC (control de acceso basado en roles)**: cada endpoint sensible exige explícitamente un rol (`admin`, `auditor`, o ambos) antes de ejecutarse.

4. Implementación Técnica por Módulo

Esta sección recorre cada archivo de código relevante y explica su propósito y funcionamiento, sin descender a una lectura línea por línea.

4.1 Backend — `backend/main.py`

Es el punto de entrada de la aplicación FastAPI y concentra todos los endpoints REST. Algunos aspectos destacables de su diseño:

- `GET /api/crypto/public-key`: endpoint público (sin autenticación) que entrega la llave pública RSA de la organización. Es necesario que sea público porque el denunciante, que no inicia sesión, la necesita para cifrar su reporte.
- `POST /api/reports/submit`: recibe el reporte ya cifrado (texto, IV, llave AES envuelta) y sus adjuntos, también cifrados. El backend nunca valida ni interpreta ese contenido: simplemente lo persiste tal cual llega y registra el evento en la bitácora. No requiere autenticación, preservando el anonimato del denunciante.
- `POST /api/auth/login`: autenticación estándar OAuth2 (usuario/contraseña por formulario). Registra tanto los intentos fallidos como los exitosos en la bitácora de auditoría, lo cual es valioso para detectar intentos de fuerza bruta.
- `GET /api/reports` y `GET /api/reports/{id}`: exclusivos para auditor/admin (vía `RoleChecker`). Devuelven los reportes **todavía cifrados** — el descifrado ocurre después, en el navegador del auditor.
- `GET /api/audit/logs`: recalcula y devuelve el estado de integridad de toda la cadena de hashes junto con el detalle de cada registro.
- `POST /api/audit/simulate-tampering`: endpoint deliberadamente “malicioso”, reservado a administradores, que modifica el campo `details` del último log **sin** recalcularse su hash. Existe con fines pedagógicos: permite demostrar en vivo que el sistema de verificación detecta la alteración.
- `POST /api/admin/users` y `GET /api/admin/users`: alta y listado de usuarios, protegidos exclusivamente para el rol `admin`.
- `GET /api/admin/backup` y `POST /api/admin/restore`: descarga y restauración directa del

archivo SQLite completo. Esto es seguro precisamente porque todo el contenido sensible ya está cifrado dentro de la base de datos — un respaldo del archivo `.db` no expone ningún reporte en texto plano.

Al final del archivo, `StaticFiles` monta el directorio `frontend/` como raíz del sitio, de modo que el mismo proceso Uvicorn sirve tanto la API como la interfaz web.

4.2 Backend — `backend/database.py`

Define el esquema de datos con SQLAlchemy ORM (lo que también previene inyecciones SQL de forma nativa, al parametrizar automáticamente las consultas) y contiene la lógica de la bitácora encadenada.

Modelos principales:

- **User**: usuario del sistema con `password_hash` (Argon2id) y `role`.
- **Report**: una denuncia. Solo almacena campos cifrados en Base64: `report_text_encrypted`, `report_iv`, `encrypted_aes_key`. En ningún momento existe una columna de texto plano.
- **Attachment**: un archivo adjunto, igualmente cifrado, vinculado a un **Report** por clave foránea.
- **AuditLog**: cada evento del sistema, con `previous_hash` y `current_hash`.

Las funciones clave son `compute_log_hash()` (calcula el SHA-256 determinista de un registro concatenando sus campos con el hash anterior), `add_audit_log()` (inserta un nuevo registro encadenándolo al último existente) y `verify_audit_trail_integrity()` (recorre toda la tabla desde el principio, recalcula cada hash y determina si la cadena sigue intacta, devolviendo además el detalle registro por registro para mostrarlo en la interfaz del administrador).

4.3 Backend — `backend/auth.py`

Concentra toda la lógica de identidad: hash y verificación de contraseñas con Argon2id (`get_password_hash`, `verify_password`), emisión de JWT firmados con expiración de 30 minutos (`create_access_token`), extracción y validación del token en cada petición protegida (`get_current_active_user`), y la clase `RoleChecker`, una dependencia de FastAPI parametrizable que se usa como `Depends(RoleChecker(["admin"]))` para exigir un rol específico en un endpoint con una sola línea.

Un detalle a señalar honestamente: la llave secreta usada para firmar los JWT (`SECRET_KEY`) tiene un valor por defecto codificado en el propio archivo si no se define la variable de entorno `SECRET_KEY`. Es una práctica aceptable para un proyecto académico ejecutado en local, pero en cualquier despliegue real debería generarse aleatoriamente y gestionarse fuera del código fuente.

4.4 Backend — `backend/crypto_utils.py`

Contiene dos responsabilidades bien diferenciadas:

1. **ensure_organizational_keys()**: en el primer arranque, genera el par de llaves RSA de 2048 bits con la librería `cryptography`, guarda la llave pública en `backend/keys/org_public.pem`, y **nunca guarda la llave privada como archivo persistente**: la pasa directamente a la función de fragmentación y descarta la referencia.
2. **split_secret_shamir()** y **gf256_mul()**: la implementación de Shamir's Secret Sharing descrita en la sección 3.2, escrita a mano (aritmética en GF(256) con el polinomio irreducible $x^8+x^4+x^3+x^2+1$) en vez de depender de una librería externa — una decisión razonable en un proyecto académico, porque demuestra comprensión real del algoritmo, aunque en producción sería preferible una implementación auditada por terceros.

4.5 Backend — `backend/ssl_gen.py`

Genera un certificado X.509 autofirmado y su llave privada para servir la aplicación por HTTPS en `localhost`. Esto no es un capricho: la Web Crypto API del navegador (`window.crypto.subtle`), que es el corazón de todo el cifrado del lado del cliente, **se niega a funcionar en un contexto no seguro** (HTTP simple, salvo `localhost` sin TLS explícito en algunos navegadores). Generar TLS local propio, aunque genere una advertencia de certificado no confiable en el navegador, es la forma más directa de garantizar que la API criptográfica esté disponible.

4.6 Orquestador — `run.py`

Es el script de arranque único del proyecto y cumple varias funciones de aprovisionamiento en secuencia: comprueba e instala dependencias de `requirements.txt` si faltan, genera los certificados TLS, inicializa las tablas de la base de datos, genera (o detecta que ya existen) las llaves RSA organizacionales y sus fragmentos Shamir, crea los usuarios semilla `admin` y `auditor` con contraseñas de demostración, inserta el log “génesis” de la bitácora, y finalmente levanta Uvicorn sirviendo la aplicación FastAPI por HTTPS en el puerto 8000.

Este diseño de “un solo comando para levantar todo” reduce fricción en un contexto de evaluación académica, donde se necesita que el sistema funcione de inmediato en cualquier máquina.

4.7 Frontend — `frontend/js/crypto_client.js`

Este es, en términos de exigencia técnica, el módulo más importante del proyecto: aquí vive **toda** la criptografía del lado del cliente, construida directamente sobre la Web Crypto API nativa del navegador (sin librerías externas de criptografía en JavaScript).

Funciones principales:

- `generarLlaveAES()`: crea una llave AES-GCM de 256 bits nueva, marcada como exportable (necesario para poder envolverla después con RSA).
- `cifrarTexto()` / `cifrarArchivoBytes()`: cifran texto o bytes de archivo con AES-GCM, generando en cada llamada un vector de inicialización (IV) aleatorio de 12 bytes — un IV distinto por cada operación es indispensable en AES-GCM; reutilizarlo compromete la seguridad del esquema.
- `importarLlavePublicaRSA()` / `importarLlavePrivadaRSA()`: convierten una llave PEM (texto) en un objeto `CryptoKey` utilizable por la API, tras limpiar cabeceras, pies y saltos de línea del formato PEM y decodificar el Base64 resultante a binario (formatos SPKI para la pública, PKCS#8 para la privada).
- `envolverLlaveAES()` / `desvolverLlaveAES()`: implementan respectivamente el cifrado y descifrado de la llave AES de sesión con RSA-OAEP — el corazón del esquema híbrido descrito en 3.1.
- `descifrarTexto()` / `descifrarArchivoBytes()`: el proceso inverso, usado en el panel del auditor una vez que la llave AES ha sido recuperada.
- `gf256_mul()`, `gf256_inv()` y `reconstruirSecretShamir()`: replican en JavaScript, en el navegador, la misma aritmética de Shamir’s Secret Sharing implementada en Python en el backend. Esta función parsea los archivos `.share` cargados por el auditor (con el formato `SD-SHARE-v1-[id]-[datos]`), calcula los coeficientes de Lagrange para `x=0` y reconstruye byte a byte la llave privada RSA original en memoria — que nunca sale del navegador ni se transmite al servidor.

Es importante notar que la reconstrucción de la llave privada ocurre **enteramente en el cliente**: el servidor jamás recibe ni un fragmento ni la llave reconstruida.

4.8 Frontend — `frontend/js/app_client.js`

Es el controlador de interfaz (unas 1350 líneas) que conecta el DOM con la lógica criptográfica anterior y con la API REST. Se organiza en tres controladores independientes, cada uno activado según la página HTML donde se cargue:

- `initSubmissionPortal()` (para `index.html`): gestiona el formulario de denuncia, el área de arrastrar-y-soltar archivos, y sobre todo la secuencia completa de envío: genera la llave AES, cifra texto y adjuntos, obtiene la llave pública del servidor, envuelve la llave AES con RSA, y finalmente envía todo por `POST /api/reports/submit`. La interfaz muestra un overlay animado con los pasos del proceso criptográfico en tiempo real, lo cual tiene valor tanto de UX como pedagógico (el usuario ve, literalmente, que su información se cifra antes de salir de su equipo).
- `initAuditorDashboard()` (para `auditor.html`): controla el login, la carga de la lista de reportes cifrados, la selección de un reporte individual (mostrando los bloques cifrados tal cual están en la base de datos), la zona de carga de fragmentos `.share` mediante arrastrar-y-soltar, y el botón de descifrado local que orquesta: reconstrucción de la llave privada vía Shamir → desenvolver la llave AES vía RSA → descifrar texto y adjuntos vía AES-GCM.
- `initAdminDashboard()` (para `admin.html`): gestiona el login de administrador, la creación de usuarios, la visualización de la línea de tiempo de la bitácora, el botón “Verificar Integridad” (que llama a `/api/audit/logs` y colorea en verde o rojo según el resultado) y el botón “Simular Ataque DB” (que llama al endpoint de tampering y permite observar en vivo cómo se rompe la cadena de hashes).

Nota de consistencia: el README original describe el flujo del auditor como una carga directa del archivo `organizacion_llave_privada.pem`. Sin embargo, el código implementado (`crypto_client.js`, `app_client.js` y `run.py`) usa consistentemente el esquema de fragmentos Shamir (`.share`), y la llave `.pem` completa se elimina intencionalmente tras generarse. Este informe describe el comportamiento real del código, que es el esquema de fragmentos 2-de-3 — más seguro y coherente con el resto del diseño del proyecto.

4.9 Frontend — `frontend/simulator.html` y `simulator.js`

Un laboratorio interactivo independiente del flujo real de la aplicación, pensado para fines didácticos y de sustentación del proyecto. Permite manipular, en sandboxes separados: el cifrado híbrido paso a paso, la partición y reconstrucción de Shamir’s Secret Sharing (incluyendo una visualización gráfica de la interpolación de Lagrange), y el encadenamiento de hashes tipo blockchain. Es una herramienta de comunicación del diseño, no parte de la ruta crítica de seguridad del sistema.

4.10 Utilidades adicionales

- `verify_system.py`: un test de integración que actúa como un cliente externo — realiza login, envía una denuncia cifrada real usando `cryptography` en Python (no JavaScript), reconstruye la llave privada a partir de shares y verifica que el descifrado funcione, y finalmente dispara el endpoint de simulación de ataque para confirmar que la verificación de integridad lo detecta. Sirve como prueba automatizada de que todo el pipeline criptográfico funciona de extremo a extremo sin depender del navegador.
- `inspect_db.py`: utilidad de línea de comandos para volcar el contenido crudo de la base de datos SQLite, útil para comprobar en la práctica que lo almacenado es efectivamente ilegible sin las llaves adecuadas.
- `bandit.yaml`: configuración del analizador estático de seguridad Bandit, usado para escanear el backend en busca de patrones de código inseguro conocidos (credenciales hardcodedas, uso de funciones peligrosas, etc.).

5. Flujos de Uso Completos

5.1 Flujo del denunciante

1. El navegador abre `https://localhost:8000` y establece una sesión TLS (aceptando la advertencia de certificado autofirmado, propia de un entorno local).
2. El usuario redacta el reporte y adjunta archivos opcionales, sin necesidad de crear cuenta ni proporcionar ningún dato identificable.
3. Al enviar, el navegador genera una llave AES-256 de un solo uso, cifra el texto y cada adjunto, obtiene la llave pública RSA del servidor, envuelve la llave AES con esa llave pública, y solo entonces transmite el paquete cifrado.
4. Inspeccionando la pestaña de Red del navegador se puede verificar que el cuerpo de la petición `POST /api/reports/submit` contiene únicamente Base64 ilegible — ningún fragmento de texto plano abandona el equipo del denunciante.

5.2 Flujo del auditor

1. Inicia sesión con sus credenciales (`auditor` / `SafeDropAuditor2026!`) en el entorno de demostración).
2. Selecciona un reporte de la lista; la interfaz muestra los bloques tal como están almacenados: la llave AES envuelta y el texto cifrado, ambos ilegibles.
3. Carga al menos dos de los tres archivos `.share` generados durante la inicialización del sistema.
4. Al pulsar “Descifrar”, el navegador reconstruye la llave privada RSA en memoria mediante interpolación de Lagrange, la usa para desenvolver la llave AES de ese reporte específico, y con ella descifra el texto y los adjuntos — todo dentro del propio navegador, sin que la llave privada reconstruida viaje jamás al servidor.

5.3 Flujo del administrador

1. Inicia sesión (`admin` / `SafeDropAdmin2026!`).
2. Pulsa “Verificar Integridad”: el sistema recalcula toda la cadena de hashes SHA-256 y muestra “Integridad Asegurada” si todo coincide.
3. Pulsa “Simular Ataque DB”: se altera intencionalmente el último registro de la bitácora sin recalculár su hash, para fines de demostración.
4. Al volver a pulsar “Verificar Integridad”, el sistema detecta la discrepancia y muestra una alerta visual señalando exactamente qué registro fue comprometido.

6. Modelo de Amenazas y Riesgos

Riesgo	Entidad afectada	Severidad	Mitigación implementada
Fuga de la base de datos completa	Denunciante, organización	Alta	Todo el contenido almacenado son blobs cifrados; sin las llaves, son ilegibles
Robo o pérdida de la llave privada RSA	Organización, auditoría	Crítica	La llave nunca existe completa en el servidor; se requiere colusión de al menos 2 de 3 custodios de fragmentos
Administrador malicioso que altera la bitácora	Auditoría, cumplimiento	Media	El encadenamiento SHA-256 hace matemáticamente detectable cualquier alteración retroactiva
Interceptación de tráfico en la red (sniffing / MITM)	Denunciante	Alta	TLS/HTTPS obligatorio en todas las comunicaciones
Fuga indirecta de identidad vía metadatos (IP, User-Agent)	Denunciante	Media	La API no registra direcciones IP ni cabeceras identificatorias

Consideraciones que el propio equipo reconoce como limitaciones

- **Memoria del navegador con archivos grandes:** el descifrado actual carga el archivo completo en un `ArrayBuffer` en memoria. Con adjuntos muy grandes (por ejemplo, video de cientos de MB), esto podría agotar la memoria del navegador; una versión de producción debería usar cifrado por flujos (*streaming*).
- **SECRET_KEY de JWT con valor por defecto:** aceptable para desarrollo local, pero debe generarse aleatoriamente y gestionarse como secreto en cualquier entorno real.
- **Certificado TLS autofirmado:** es la opción correcta para un entorno de demostración local, pero en producción se necesitaría un certificado emitido por una autoridad de certificación reconocida.
- **Custodia física de los fragmentos Shamir:** el esquema 2-de-3 es tan seguro como la disciplina operativa con la que se repartan y resguarden los tres archivos `.share`; si los tres terminan en el mismo lugar, se pierde la ventaja de la separación de privilegios.

7. Plan de Respuesta ante Incidentes (resumen)

El proyecto contempla procedimientos de respuesta para tres escenarios:

1. **Fuga de la base de datos:** contener deteniendo el servidor, auditar el vector de acceso, confirmar que los datos filtrados siguen cifrados (mitigando el impacto real), y forzar el cambio de credenciales de todos los usuarios.
2. **Compromiso de la llave privada (o de sus fragmentos):** revocar la llave pública activa (las nuevas denuncias usarán un par nuevo), y para las denuncias históricas, descifrarlas en un entorno aislado con la llave comprometida y re-cifrarlas con el nuevo par.
3. **Corrupción de la bitácora:** usar la propia cadena de hashes para localizar el punto exacto de la manipulación, y restaurar desde el último respaldo verificado.

8. Consideraciones Éticas

El diseño del sistema incorpora principios de *privacy by design*: no exige registro ni datos personales del denunciante, minimiza deliberadamente los metadatos que la API podría capturar, y traslada la protección de la identidad del ámbito de la política organizacional (que puede incumplirse) al ámbito de una restricción matemática (que no puede incumplirse sin romper la criptografía). Esta es, en esencia, la tesis central del proyecto: la privacidad de un informante no debería depender de la promesa de un administrador, sino de una garantía verificable.

9. Análisis de Seguridad Estática (Bandit)

El backend fue analizado con Bandit, una herramienta de análisis estático (SAST) para Python orientada a detectar patrones de código inseguro. Según el propio proyecto, el escaneo no reportó vulnerabilidades, apoyado en tres factores: el uso de SQLAlchemy ORM previene inyecciones SQL de forma nativa al parametrizar automáticamente las consultas; las contraseñas se almacenan exclusivamente con Argon2id; y los algoritmos criptográficos usados (AES-GCM-256, RSA-OAEP-2048 con SHA-256) están alineados con las recomendaciones vigentes del NIST.

10. Instrucciones de Ejecución

Requisitos: Python 3.10 o superior, y un navegador moderno con soporte de Web Crypto API (Chrome, Edge o Firefox recientes).

1. Ejecutar `python run.py` desde la raíz del proyecto. El script instala dependencias faltantes, genera certificados TLS, inicializa la base de datos y las llaves, y levanta el servidor.
 2. Guardar en un lugar seguro los tres archivos `llave_privada_compartida_*.share` generados en la raíz — son necesarios (dos de los tres) para descifrar denuncias como auditor.
 3. Abrir `https://localhost:8000` y aceptar la advertencia de certificado autofirmado (esperada en un entorno local de desarrollo).
 4. Credenciales de demostración: administrador `admin` / `SafeDropAdmin2026!`; auditor `auditor` / `SafeDropAuditor2026!`.
 5. Opcionalmente, ejecutar `python verify_system.py` en otra terminal para correr el test de integración automatizado de extremo a extremo.
-

11. Recomendaciones para una Evolución hacia Producción

1. **Gestión de fragmentos multi-custodio en la nube:** automatizar la distribución de los tres fragmentos Shamir en servicios independientes de gestión de llaves (AWS KMS, Azure Key Vault, Google Cloud KMS), en vez de archivos locales.
2. **Módulos de seguridad de hardware (HSM):** almacenar la llave organizacional en un chip criptográfico inexportable, combinado con autenticación física multifactor (por ejemplo, YubiKeys) para autorizar el descifrado.
3. **Cifrado por flujos para adjuntos grandes,** evitando cargar archivos completos en memoria del navegador.
4. **Gestión de secretos fuera del código fuente** (la `SECRET_KEY` de JWT, en particular) mediante variables de entorno o un vault dedicado.
5. **Certificados TLS emitidos por una autoridad reconocida** en cualquier despliegue

accesible fuera de `localhost`.

12. Conclusiones

SafeDrop Local demuestra, con una implementación funcional y no solo teórica, cómo la criptografía aplicada puede convertir un principio ético abstracto —proteger al informante— en una garantía técnica verificable. El proyecto no se limita a cifrar datos: articula tres mecanismos independientes (cifrado híbrido, reparto de secretos y bitácora encadenada) que juntos eliminan la necesidad de que el denunciante confíe ciegamente en la organización, el servidor o un único custodio de llaves. Las limitaciones identificadas por el propio equipo son honestas y coherentes con lo esperado de un proyecto académico: son ajustes de ingeniería hacia un entorno de producción, no fallas conceptuales del diseño de seguridad.